

Parallelization in `loo` (current state)

Stan GSOC 2026

Table of contents

1	Background	1
2	Approaches to parallelization	2
3	Introducing <code>mirai</code> for parallelization in <code>loo</code>	3
3.1	How does <code>mirai</code> work?	3
3.2	Q1: Can we use <code>mirai</code> for serial execution?	7
3.3	What does going through <code>future</code> cost?	8
4	Memory	12
4.1	Memory usage and <code>loo</code> (Why does it matter?)	12
4.2	Shared memory & the potential of <code>mori</code>	12
4.3	Q2: Can we use <code>mori</code> for sharing model methods?	12
5	Q3: Where is parallelization currently in use in <code>loo</code> ?	16
6	Q4: Where could we benefit from introducing parallelization in <code>loo</code> ?	17
6.1	Analytic search	17
6.2	Vectorization	18
7	Nested parallelization	22
7.1	Q5: Where do we currently find nested parallelization in <code>loo</code> ?	22
7.2	Can packages that use <code>loo</code> cause nested parallelization?	23

i Guiding questions

- Q1: What is the effect of using `mirai` for serial execution (Section 3.2)?
- Q2: Can we use `mori` for sharing model methods (Section 4.3)?
- Q3: Where is parallelization currently in use in `loo` (Section 5)?
- Q4: Where could we benefit from introducing parallelization in `loo` (Section 6)?
- Q5: Where do we currently find nested parallelization in `loo` (Section 7.1)?

1 Background

Please refer to the appendix for definitions of some important terms.

Currently `loo` relies on the `parallel` package from base R [1] to parallelize certain computations, such as moment matching, Pareto smoothed importance sampling [2], and more. This works well on Unix systems, where `parallel` can use forking, but Windows requires a separate, socket-based path. This project will replace `loo`'s current parallelization structure, swapping to a platform-agnostic approach using `mirai` [3]. This will have the added benefit of allowing users to parallelize

execution over heterogeneous compute resources (e.g., over local cores, a distributed system, GPUs, or any mixture). This freedom decouples execution profiles from `loo`'s software, allowing users with varying compute needs to exploit their available resources without any extra work. Further, a major part of the project here would be writing strong documentation which teaches users how to make use of some common resources such as SLURM or SSH access, building off of the documentation in packages like `mirai`, but with a focus on applications to `loo`.

This report is a survey of where `loo` stands before any of that work lands. Section 2 lays out the four maps `loo` could dispatch work through and what each one costs. The next section covers why we picked `mirai`, how it behaves with respect to RNG, and what we pay for using it in place of `lapply()` when a user asks for serial execution (Section 3.2). Section 4 covers why memory is a concern when parallelizing, and whether `mor` can help (Section 4.3). The remaining sections are an audit of the package itself: which functions parallelize today (Section 5), which ones should but don't (Section 6), and where `loo` already has to guard against nesting one parallel call inside another (Section 7.1).

2 Approaches to parallelization

Currently, parallelization in `loo` goes through [paths 1-3](#) [4]:

1. Serial execution using `lapply()`
 - run one task after the other
 - no parallelization; synchronous
2. Forking through `parallel::mclapply()`
 - create child processes as a copy of the parent's memory that run in parallel, but the main R session blocks and waits for all of them to finish before returning the result list
 - parallel, synchronous
 - not available in Windows: falls back to `lapply()`
3. Fresh processes through `parallel::parLapply()`
 - start worker processes as completely fresh R sessions, communicating with the controller over sockets. No shared memory which is in contrast to forking.
 - parallel, synchronous
4. Parallelization with `mirai_map()` (**new**)
 - dispatch tasks to persistent background R processes (daemons) over NNG sockets; workers stay alive between calls and are reused across tasks
 - allows for asynchronous parallelization with persistent workers

Comparison between approaches

	<code>lapply()</code>	<code>mclapply()</code>	<code>parLapply()</code>	<code>mirai_map()</code>
Parallel	no	yes	yes	yes

	<code>lapply()</code>	<code>mclapply()</code>	<code>parLapply()</code>	<code>mirai_map()</code>
Blocking	yes	yes	yes	yes (with <code>[]</code>)
Mechanism	in-process	forking	cluster (often PSOCK sockets)	NNG daemons
Shared memory	N/A	yes (copy-on- write)	no	no
Windows	yes	no	yes	yes
Persistent work- ers	no	no	yes	yes
Remote Execu- tion	no	no	yes	yes
Export data	no	no	yes	yes
Setup needed	none	none	<code>makeCluster</code>	<code>daemons()</code>

Thus, `mirai` works for all OS and allows for asynchronous parallelization. The only major downside of `mirai` is that it does not provide a possibility for workers to share memory (however, when using `mirai` with only local daemons, one can use `mori` (see Section 4.2) [5]).

3 Introducing `mirai` for parallelization in `loo`

The parallelization package we are using is `mirai` [3], self-described as a “Minimalist Async Evaluation Framework for R”. Before we briefly describe how `mirai` works at a high level, it is useful to mention some of its benefits which led us to select it as our parallel backend.

`mirai` very naturally allows for heterogeneous compute through a clean API. What this means in practice is that, once we migrate to `mirai`, users will be able to run `loo` code in parallel on their laptop, on remote computers, on an HPC system, or any zany mixture ([3], specifically see [sec. 6](#) in the Reference Manual). Decoupling the execution profile from `loo` code, especially using the API `mirai` exposes, allows users to exploit their available computational resources without much hassle.

`mirai` is also [widely used](#) across major R packages—for example, recently landing in `purrr` ([6], [7], see [in_parallel\(\)](#)). `mirai` is also “the first official alternative communications backend for base R’s `parallel` package” [3], and is used in other major packages like `shiny` ([3], see [the vignette](#)), `crew` ([8], see the [README](#)), and in `tune`, which is a vital part of the `tidymodels` workflow ([9], see [the vignette](#)).

3.1 How does `mirai` work?

Note that `mirai` is driven by `nanonext` [10], which binds NNG (Nanomsg Next Gen), a C messaging library.

NNG works by having a dispatcher which daemons call in to. `daemons(4)` on a laptop spawns four background R processes and hands each one an IPC address to connect back on, whereas `daemons(url = host_url())` opens the listener and waits, leaving the user free to start `daemon()` processes wherever they like. Since the connection is always initiated by the daemon, only the host needs a routable address, and going from local cores to a compute node changes the URL and nothing else. This is what allows `mirai`, and thus `loo`, to be so flexible about where computations are run—the host process needs to know very little about each daemon. Tasks travel over those connections as serialized R objects, and daemons block. NNG hands over a result as soon as it lands, so an idle daemon costs nothing and there is no supervising loop running in R. This is what makes `mirai()` asynchronous: what it returns is a placeholder and `[]` (see the glossary) is the blocking mechanism to actually collect results.

By default `daemons()` also starts a dispatcher, which queues tasks and hands each one off as a daemon frees up, giving first-in-first-out (FIFO) scheduling instead of the fixed round-robin assignment used when `dispatcher = FALSE` (which is also what `parLapply()` essentially does). This matters wherever task costs are uneven. The dispatcher is also what `stop_mirai()` and `.timeout` rely on to cancel a task that is already running, and it is where the `memory` argument blocks new tasks until the bytes queued for dispatch drop under a cap.

3.1.a Reproducibility with `mirai`

One thing worth discussing for a moment is random number generation in `mirai`, which uses the L'Ecuyer PRNG. The details around RNG in `mirai` are pretty straightforward, as we can see through a few very simple examples.

Firstly, even if we set the same seed, we won't get the same results from `lapply()` and `mirai_map()`. Here, we can see that even if we use the same seed (`set.seed(0)` before each call), randomly generating a few numbers from an exponential distribution changes depending on whether we call `lapply()` or `mirai_map()`. More specifically, both maps are essentially doing `rexp(1)`, `rexp(2)`, `rexp(3)`, `rexp(4)`.¹ In this snippet we are using 4 daemons, so in the `mirai_map()` call, we are starting 4 RNG streams which are being derived from the main stream (see [in source](#)).

```
set.seed(0)
str(lapply(1:4, rexp))
```

```
List of 4
 $ : num 0.184
 $ : num [1:2] 0.146 0.14
 $ : num [1:3] 0.436 2.895 1.23
 $ : num [1:4] 0.54 0.957 0.147 1.391
```

¹Somewhat orthogonal to this section, but note that `with(set.seed(0), lapply(1:4, rexp) |> unlist())` is equivalent to `with(set.seed(0), rexp(10))`.

```
# using with() just for scoping
with(
  {
    set.seed(0) # set seed in main process
    mirai::daemons(4) # start 4 background processes
  },
  # mirai_map() is an async map
  # `[ ]` blocks to collect results
  str(mirai::mirai_map(1:4, rexp)[])
)
```

```
List of 4
$ : num 0.809
$ : num [1:2] 0.63 0.285
$ : num [1:3] 0.851 0.252 0.779
$ : num [1:4] 1.603 0.705 0.249 0.618
```

Note that by default if `daemons()` is called without setting a seed, the RNG streams aren't reproducible (just like results in serial execution without setting a seed)—ensuring “statistical validity but not numerical reproducibility between runs” [11]. If we rerun the exact same `mirai` code, we do get the same results:

```
with(
  {
    set.seed(0)
    mirai::daemons(4)
  },
  str(mirai::mirai_map(1:4, rexp)[])
)
```

```
List of 4
$ : num 0.809
$ : num [1:2] 0.63 0.285
$ : num [1:3] 0.851 0.252 0.779
$ : num [1:4] 1.603 0.705 0.249 0.618
```

With 2 daemons, `mirai_map(1:4, rexp)` still creates four tasks: `rexp(1)`, `rexp(2)`, `rexp(3)`, `rexp(4)`. The first two tasks run on the two available daemons (so they have the same results as before); later tasks are assigned as daemons become free. Since RNG streams are tied to daemons, changing the number of daemons changes which stream each task uses, so the output changes. A plain `set.seed(0)` is just seeding the host session and making each daemon derive its RNG state from host.

```
with(
  {
    set.seed(0)
    mirai::daemons(2)
  },
  str(mirai::mirai_map(1:4, rexp)[])
)
```

```
List of 4
 $ : num 0.809
 $ : num [1:2] 0.63 0.285
 $ : num [1:3] 0.636 1.804 0.756
 $ : num [1:4] 3.7305 0.731 0.0452 0.1211
```

Setting a seed manually when launching daemons gets reproducibility (stability over runs), “regardless of the number of daemons used.” [11]. Underneath the hood, `mirai` creates a new deterministic RNG stream per `mirai()` evaluation, not per daemon, which ensures that results are stable irrespective of daemon count.

```
with(
  {
    mirai::daemons(4, seed = 0)
  },
  str(mirai::mirai_map(1:4, rexp)[])
)
```

```
List of 4
 $ : num 1.02
 $ : num [1:2] 0.0931 0.5214
 $ : num [1:3] 2.314 0.373 1.618
 $ : num [1:4] 0.821 0.886 3.642 1.572
```

```
with(
  {
    mirai::daemons(3, seed = 0)
  },
  str(mirai::mirai_map(1:4, rexp)[])
)
```

```
List of 4
 $ : num 1.02
```

```
$ : num [1:2] 0.0931 0.5214
$ : num [1:3] 2.314 0.373 1.618
$ : num [1:4] 0.821 0.886 3.642 1.572
```

In essence, reproducibility is the users' responsibility, akin to compute setups. This further stresses the fact that documentation is a major part of this project.

3.2 Q1: Can we use **mirai** for serial execution?

Since **mirai** [3] neatly subsumes paths 2 and 3, a question we had was what would the performance hit be if we also removed 1—that is, if a user requested serial execution, what do we lose by using **mirai** with a single core.

```
log_ratios <- loo::example_loglik_matrix()
S <- nrow(log_ratios)
N <- ncol(log_ratios)
tail_len <- loo::n_pareto(r_eff = rep(1, N), S = S)

idx <- seq_len(N)

work <- function(i) {
  loo::do_psis_i(
    log_ratios_i = log_ratios[, i],
    tail_len_i = tail_len[i]
  )$pareto_k
}

mirai::daemons(1)

results <- bench::mark(
  vapply = vapply(idx, work, numeric(1)),
  lapply = lapply(idx, work),
  mirai = mirai::mirai_map(
    idx,
    work,
    log_ratios = log_ratios,
    tail_len = tail_len
  )[],
  vapply_to_list = vapply(idx, work, numeric(1)) |> as.list(),
  purrr_map = purrr::map_dbl(idx, work),
  check = same_numeric_values,
  iterations = 200
)
```

Note that `mirai_map()` and `lapply()` results are lists. `same_numeric_values()`'s definition is elided from output but simply checks that results are identical without counting unlisting as part of the benchmark (unlike `vapply_to_list`).

```
results |> summary()
```

```
# A tibble: 5 × 6
  expression      min  median `itr/sec` mem_alloc `gc/sec`
  <bch:expr>    <bch:tm> <bch:tm>    <dbl>   <bch:byt>   <dbl>
1 vapply        12.3ms   19ms      47.2    3.76MB     7.68
2 lapply        12.8ms  25.2ms     38.8    3.44MB     6.31
3 mirai         25.4ms  35.4ms     25.8  586.89KB     0.393
4 vapply_to_list 11.9ms  18.3ms     47.4    3.44MB     8.04
5 purrr_map     12.2ms  17.7ms     52.1    4.2MB      8.49
```

```
results |> summary(relative = TRUE)
```

```
# A tibble: 5 × 6
  expression      min median `itr/sec` mem_alloc `gc/sec`
  <bch:expr>    <dbl> <dbl>    <dbl>   <dbl>   <dbl>
1 vapply        1.03  1.08     1.83     6.56    19.5
2 lapply        1.07  1.43     1.50     6.00    16.0
3 mirai         2.12  2.00     1       1       1
4 vapply_to_list 1      1.03     1.84     6      20.4
5 purrr_map     1.02  1       2.02     7.33    21.6
```

NB: the `mem_alloc` column should not be interpreted as total memory usage for `mirai`: `bench::mark()` records R heap allocations via `utils::Rprofmem()`, which is not able to track daemons' memory usage [1], [3], [12].

`mirai_map()` with a single daemon has a median runtime of about 30 ms, roughly half the speed of the fastest in-process approach. Of course, in absolute terms the difference is negligible, adding only about 20 ms. The memory usages of all in-process approaches are relatively close to one another. `vapply()`'s small memory overhead probably can be attributed to the extra work it does validating types and lengths and such [1].

In short, it is probably a good idea to directly test the effects of serial `mirai` execution in `loo`, but it is likely that we will find that we should keep the unique single-core execution path.

3.3 What does going through `future` cost?

`future` [13] is the other general parallelization framework in R, sitting behind `future.apply` [14] and, through `futurize()`, most of the other map-reduce APIs [15]. `future.mirai` [16] lets `future`

run on `mirai` daemons, so `loo` could dispatch through `future` instead and let users pick a backend with `plan()`, without giving up the remote and HPC execution `mirai` already gets us.

This is the same PSIS task as Section 3.2, now over four workers instead of one.

```
mirai::daemons(4)
future::plan(future.mirai::mirai_cluster)

results <- bench::mark(
  lapply = lapply(idx, work),
  mirai_map = mirai::mirai_map(
    idx,
    work,
    log_ratios = log_ratios,
    tail_len = tail_len
  )[],
  future_mirai = future.apply::future_lapply(idx, work),
  check = TRUE,
  iterations = 200
)

mirai::daemons(0)
future::plan(future::sequential)

summary(results)
```

```
# A tibble: 3 × 6
  expression      min  median `itr/sec` mem_alloc `gc/sec`
  <bch:expr> <bch:tm> <bch:tm>    <dbl> <bch:byt>    <dbl>
1 lapply        5.55ms  6.5ms    149.    5.12MB    11.2
2 mirai_map     5.05ms  8.06ms    124.   731.22KB     3.83
3 future_mirai  30.29ms  46.14ms    22.7    5.12MB    13.6
```

```
summary(results, relative = TRUE)
```

```
# A tibble: 3 × 6
  expression      min median `itr/sec` mem_alloc `gc/sec`
  <bch:expr> <dbl> <dbl>    <dbl>    <dbl>    <dbl>
1 lapply        1.10   1      6.57     7.17     2.93
2 mirai_map      1     1.24   5.45     1        1
3 future_mirai  6.00   7.10   1        7.17     3.56
```

`mirai_map()` has a median runtime of about 8 ms, about 1.25x slower than `lapply()`—this is not surprising with tasks this small. `future_lapply()` over the same daemons takes about 45 ms and is about 5.5x slower than `mirai_map()`, and slower than not parallelizing at all.

Removing the work isolates overhead so we can see what using `mirai` through `future` costs.

```
mirai::daemons(4)
future::plan(future::mirai::mirai_cluster)

results <- bench::mark(
  mirai_map = mirai::mirai_map(idx, identity)[],
  future_mirai = future::future_lapply(idx, identity),
  check = TRUE,
  iterations = 200
)

mirai::daemons(0)
future::plan(future::sequential)

summary(results)
```

```
# A tibble: 2 × 6
  expression      min   median `itr/sec` mem_alloc `gc/sec`
  <bch:expr>  <bch:tm> <bch:tm>    <dbl>  <bch:byt>    <dbl>
1 mirai_map      1.67ms  2.24ms    420.    587KB      4.24
2 future_mirai  27.08ms  31.17ms    31.2    224KB      4.25
```

```
summary(results, relative = TRUE)
```

```
# A tibble: 2 × 6
  expression      min median `itr/sec` mem_alloc `gc/sec`
  <bch:expr>  <dbl> <dbl>    <dbl>    <dbl>    <dbl>
1 mirai_map      1      1    13.5     2.62      1
2 future_mirai  16.2   13.9      1        1     1.00
```

Here we can see that pure `mirai` is about an order of magnitude faster than going through `future`. This is in part due to the ergonomics of `future.apply` as the package is designed to be user-friendly and not as low-level as `mirai_map()`, so this result was broadly expected. For `loo`, it seems appropriate to take the performance uplift at the cost of a lower-level API for parallel mapping.

3.3.a Dispatching

We can see how efficient *dispatching* specifically is by constructing 16 tasks with uneven runtimes: four each taking 0.5, 1, 1.5, and 2 seconds, and comparing `mirai` and `future` against the

theoretical optimum; with four workers, that would be $\frac{2+4+6+8}{4} = 5$ seconds. As mentioned in Section 3.1, `mirai` uses a FIFO scheduler which should enable close-to-optimal scheduling. Default `future.apply` does not do this, instead splitting the input into a fixed number of chunks by default, with one chunk per worker [13], [14]. By construction then, the four workers receive tasks totaling 2, 4, 6, and 8 seconds, making 8 seconds the expected runtime. Setting `future.scheduling = Inf` creates one future per element, allowing work to be assigned as workers become available and bringing the runtime much closer to the 5 second optimum, but this increases overhead due to creating many more futures [13], [14].

```
x <- rep(c(0.5, 1, 1.5, 2), each = 4)

mirai::daemons(4)
future::plan(future.mirai::mirai_cluster)

results <- bench::mark(
  mirai = mirai::mirai_map(x, Sys.sleep)[],
  future_mirai = future.apply::future_lapply(x, Sys.sleep),
  future_mirai_dispatched = future.apply::future_lapply(
    x,
    Sys.sleep,
    future.scheduling = Inf
  ),
  check = TRUE,
  iterations = 5
)

mirai::daemons(0)
future::plan(future::sequential)

summary(results)
```

```
# A tibble: 3 × 6
  expression      min median `itr/sec` mem_alloc `gc/sec`
  <bch:expr>    <bch:tm> <bch:tm>    <dbl>   <bch:byt>   <dbl>
1 mirai          5.09s   5.11s     0.196  437.33KB     0
2 future_mirai    8.2s     8.22s     0.122   5.28MB     0.146
3 future_mirai_dispatched 6.1s     6.12s     0.162  916.55KB     0.195
```

```
summary(results, relative = TRUE)
```

```
# A tibble: 3 × 6
  expression      min median `itr/sec` mem_alloc `gc/sec`
```

<bch:expr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1 mirai	1	1	1.61	1	NaN
2 future_mirai	1.61	1.61	1	12.4	Inf
3 future_mirai_dispatched	1.20	1.20	1.33	2.10	Inf

As we can see, `mirai`'s dispatching is clearly the fastest, and `future` is slower—even with `future.scheduling = Inf` which brings the total time down from 8 to about 6 seconds.

4 Memory

4.1 Memory usage and `loo` (Why does it matter?)

Parallelization may lead to running out of memory (OOM). Copying the construction from that note: if you have, say, 8 GB memory but spin up 16 tasks which each use 1 GB, you will run out of memory. The math here is quite clear, though it is worst case and is complicated by the profile of the program—if that 1 GB is peak usage and it normally doesn't go so high, and the peaks don't overlap, then it is possible for you to squeeze past without 16 GB free. Note, however, that this 1 GB usage per parallel worker is assuming that there is no memory that can be shared.

4.2 Shared memory & the potential of `mori`

`mirai` by itself does not allow for shared memory. However, `mori` is a recently released R package that allows for sharing memory across workers by placing an R object once into OS-level shared memory and letting every process on the machine read the same physical pages directly, with no data copying between processes [17].

If objects are reused across workers, we could use something like `mori` [17] in R to reduce the physical memory usage by mapping some objects in workers to the same shared address spaces, thus eliding copies—unless, of course, a worker modifies the object [5]. This can be loosely seen as mildly clawing back some of the benefits of forking with respect to memory. Note, however, that `mori` works on a limited set of R objects, specifically “atomic vector types, lists, and data frames” [17]. See Section 4.3 for empirical proof that we cannot naively use `mori` to share compiled model methods across workers.

Since OOM is a slightly tricky thing to think about, one user-friendly feature we would like to implement, somewhat orthogonal to swapping to `mirai`, is having parallel functions check how much memory they need and warn users if a run might run out of memory. We could do this pretty simply by running our functions across varying input sizes and modelling peak memory usage (probably averaged over a few runs) as a function of the parameters. Of course, if we implement this, we would need to test it to see how accurate and conservative our model is.

4.3 Q2: Can we use `mori` for sharing model methods?

No.

In this example, we build a trivial model, compile model methods, and attempt to use `mori` to share the fit. As we can see, this doesn't work—even though `theta`, a simple vector, is actually shared. This just shows that `mori` is working, but `fit` cannot be shared [17].

```

library(cmdstanr)

stan_file <- file.path(
  cmdstan_path(),
  "examples",
  "bernoulli",
  "bernoulli.stan"
)

model <- cmdstan_model(
  stan_file,
  compile_model_methods = TRUE,
  force_recompile = TRUE
)

fit <- model$sample(
  data = list(N = 1, y = 0),
  fixed_param = TRUE,
  chains = 1,
  iter_warmup = 0,
  iter_sampling = 1,
  refresh = 0,
  show_messages = FALSE
)

fit$init_model_methods()

theta <- mori::share(seq(0.01, 0.99, length.out = 100))
fit_shared <- mori::share(fit)

```

```
fit_shared |> class() |> tail(1)
```

```
[1] "R6"
```

```
mori::is_shared(fit_shared)
```

```
[1] FALSE
```

```
mori::is_shared(theta)
```

```
[1] TRUE
```

```
lp <- function(theta, fit) {  
  fit$log_prob(  
    fit$unconstrain_variables(list(theta = theta)),  
    jacobian = FALSE  
  )  
}  
  
with(  
  mirai::daemons(4),  
  mirai::mirai_map(  
    theta,  
    lp,  
    .args = list(fit = fit_shared)  
  )[] |>  
  unlist() |>  
  table()  
)
```

```
Error in .Call(<pointer: (nil)>, ext_model_ptr, variables): NULL value passed  
as symbol address
```

100

As we can see from the output of `mori::is_shared(fit)`, `fit` was not converted to a shared ALTREP form² as it is an R6 object. `mori` successfully shares the plain numeric vector `theta`, but it does not share `fit`. The later NULL pointer errors arise when daemons try to call model methods through that non-shared `fit` object.

```
shared <- mori::share(list(fit = fit, theta = theta))  
c(  
  shared = mori::is_shared(shared),  
  fit = mori::is_shared(shared$fit),  
  theta = mori::is_shared(shared$theta)  
)
```

```
shared    fit    theta  
TRUE FALSE    TRUE
```

²ALTREP is short for “alternate representation”, and “allows R to represent certain types of vectors very compactly.” [18, Sec. 2.4].

```

with(
  mirai::daemons(4),
  mirai::mirai_map(
    shared$theta,
    function(theta, shared) {
      shared$fit$log_prob(
        shared$fit$unconstrain_variables(list(theta = theta)),
        jacobian = FALSE
      )
    },
    .args = list(shared = shared)
  )[] |>
  unlist() |>
  table()
)

```

```

Error in .Call(<pointer: (nil)>, ext_model_ptr, variables): NULL value passed
as symbol address

```

100

We can see that even though we can technically use `mori::share()` on the list containing the `fit` object, it doesn't actually share `fit`.

4.3.a Can we use `mori` to share a `loo` object?

Yes. Since a `loo` object is made up of atomic data types, it can be serialized correctly.

```

loo <- loo::loo(loo::example_loglik_matrix(), save_psis = TRUE)
shared_loo <- mori::share(loo)

with(
  mirai::daemons(4),
  mirai::mirai_map(
    1:4,
    \(i, loo) loo$estimates["elpd_loo", "Estimate"],
    .args = list(loo = shared_loo)
  )[] |>
  unlist()
)

```

```
[1] -83.58923 -83.58923 -83.58923 -83.58923
```

```
loo$estimates["elpd_loo", "Estimate"]
```

```
[1] -83.58923
```

We can check that the serialization is actually working using the pattern demonstrated in the `mori` README, using `lobstr` to check the size of the object shared to each worker.

```
with(
  mirai::daemons(4),
  mirai::mirai_map(
    1:4,
    \(i, loo) lobstr::obj_size(loo) |> format(),
    .args = list(loo = shared_loo)
  )[] |>
  unlist() |>
  unique()
)
```

```
[1] "824 B"
```

```
lobstr::obj_size(loo) |> format()
```

```
[1] "263.64 kB"
```

As we can see, each worker is just getting the small ALTREP pointer (824 bytes), not the full `loo` object.

5 Q3: Where is parallelization currently in use in `loo`?

- `do_importance_sampling()`
 - The backend for `psis()`, `tis()`, and `sis()`, and by extension for `loo.matrix()` and `loo.array()`. Maps over log-ratio columns independently.
- `parallel_importance_sampling_list()`
 - Used by `loo.function()` to map `.loo_i` over observations, and by `loo_approximate_posterior.function()` through `parallel_psis_list()`. Each task evaluates the user's log-likelihood function for one observation and then performs importance sampling on the result.
- `relative_eff.array()`
 - Computes `posterior::ess_mean()` over the third dimension of the draws array, producing one relative efficiency per observation. `relative_eff.matrix()` reshapes its input and delegates here, so the matrix method is covered by the same branch.

- [relative_eff.function\(\)](#)
 - Computes the same quantity, but with likelihood values generated on demand. Its Windows branch is the only one of these seven sites to explicitly call [clusterExport\(\)](#), exporting `draws`, `chain_id`, and `data` to the workers.
- [loo_moment_match.default\(\)](#)
 - Maps over the observations whose Pareto $\hat{k}$ exceeds `k_threshold` rather than over all observations, so the task count is data-dependent and can be zero.
- [compute_lpds\(\)](#)
 - Computes per-observation log predictive densities used to construct several approximations in [loo_subsample\(\)](#). This happens over the input data before the subsample indices are constructed. It calls `mclapply()` and friends unqualified, and tests `.Platform$OS.type != "windows"` directly rather than going through the shared `os_is_windows()` helper.
- [psislw\(\)](#)
 - Deprecated in favor of `psis()` since 2.0.0, but it still carries its own copy of the parallel branch, written in the same unqualified style as `compute_lpds()`. This will be removed in `loo v3.0.0` (see the drafted [v3.0.0 migration guide](#)).

6 Q4: Where could we benefit from introducing parallelization in `loo`?

There are some functions in `loo` which could benefit from parallelization, potentially to great effect, as these functions appear to do “embarrassingly parallel” work—meaning that they do independent tasks which seem amenable to speedups by executing those tasks concurrently. These are prime targets for parallelization and somewhat easily identified; we do so by inspection, as covered in the next section.

6.1 Analytic search

We can see in [elpd_loo_approximation\(\)](#) that we don’t push `cores` through to `loo.function()` so right now, this function only parallelizes if users set the `mc.cores` option as `loo.function()` will pick this up when it’s called without specifying `cores`. This is a simple fix and could speed up some code, depending on how users tend to set their `cores`.

[Later on](#) in the same function, we can see that parallelization had been thought of but not implemented yet for the delta method approximations in that function (`waic_grad`, `waic_grad_marginal`, `waic_hess`). Each of these loop row-wise over data and shouldn’t be too difficult to parallelize. More careful thought must go into determining if parallelization is even worth it, and more careful scrutiny of the call sites of [elpd_loo_approximation\(\)](#) to ensure it doesn’t accidentally induce nested parallelization (see Section 7)—though at first blush this seems unlikely.

In [waic.function\(\)](#) `loo` simply doesn’t parallelize the work, relying only on a base `lapply()`. This seems like a pretty easy fix, and there doesn’t seem to be any reason not to use parallelization. Again, we should be careful and benchmark to ensure we aren’t accidentally eating performance or RAM, but this could be a large, free win.

Lastly, in `ap_psis.array()`, which mostly just converts array arguments to matrices before passing the buck to the matrix method, we can note that `cores` is hard-coded to 1 when calling `ap_psis.matrix()`. There doesn't seem to be any real reason to do this, eating the input `cores` like in the first example; unlike there though, we pass `cores = 1`. This is not a huge issue because users probably don't use this function, since most times they would be working directly with matrices. Further evidencing this hypothesis is [this line](#) which should error out since it references a nonexistent `r_eff`. That obviously needs to be fixed, and this function tested.

The `elpd_loo_approximation()`, `waic.function()`, and `ap_psis.array()` changes are implemented in [#371](#).

6.2 Vectorization

When browsing the codebase to find places where parallelization could be helpful (see Section 6), we found two potential speedups—in these places, we may be able to benefit from *vectorizing* functions instead of parallelizing a loop. Vectorization is advantageous in these spots as we would be rewriting R loops as optimized matrix operations [18, Secs. 24.5–24.7].

Matrix algebra is a general example of vectorisation. There loops are executed by highly tuned external libraries like BLAS. If you can figure out a way to use matrix algebra to solve your problem, you'll often get a very fast solution.

Of course, for these potential improvements, we should first write tests for correctness, then write benchmarks in the PR.

The more important of the two is in `pseudobma_weights()`, when performing a Bayesian bootstrap. This function is the backend for `loo_model_weights()`, and is used to average models using pseudo-Bayesian model averaging, optionally using a Bayesian bootstrap (which is referred to as pseudo-BMA+). This pseudo-BMA+ path is the one we care about as it has the elidable loop. We currently loop over Bayesian bootstrap iterations where it seems quite plausible to swap to matrix operations. This could be a large speedup as the loop is large, being of length `BB_n`, which defaults to 1,000 replications. There don't appear to be any reallocations in the loop, so the speedup will probably be dominated by the efficiency of matrix ops [18, Secs. 5.3.1, 24.6]. This is implemented in [#368](#).

Here is a terse benchmark isolating the main loop to see the performance uplift almost purely from swapping to matrix operations. This benchmark isn't entirely representative of the real code since we are lowering all the out-of-loop, fixed costs.

```
set.seed(0)
N <- 400
K <- 6
BB_n <- 1000
alpha <- 1
lpd_point <- matrix(rnorm(N * K, mean = -1), nrow = N, ncol = K)
```

```

BB_weighting <- loo:::dirichlet_rng(BB_n, rep(alpha, N))

pseudobma_bb_loop <- function(lpd_point, BB_weighting) {
  N <- nrow(lpd_point)
  K <- ncol(lpd_point)
  BB_n <- nrow(BB_weighting)
  temp <- matrix(NA, BB_n, K)

  for (bb in 1:BB_n) {
    z_bb <- BB_weighting[bb, ] %*% lpd_point * N
    uwts <- exp(z_bb - max(z_bb))
    temp[bb, ] <- uwts / sum(uwts)
  }

  colMeans(temp)
}

pseudobma_bb_vectorized <- function(lpd_point, BB_weighting) {
  N <- nrow(lpd_point)
  z <- BB_weighting %*% lpd_point * N
  uwts <- exp(z - matrixStats::rowMaxs(z))

  colMeans(uwts / rowSums(uwts))
}

results <- bench::mark(
  loop = pseudobma_bb_loop(lpd_point, BB_weighting),
  vectorized = pseudobma_bb_vectorized(lpd_point, BB_weighting),
  iterations = 1e3
)

summary(results)

```

```

# A tibble: 2 × 6
  expression      min   median `itr/sec` mem_alloc `gc/sec`
  <bch:expr> <bch:tm> <bch:tm>    <dbl>   <bch:byt>   <dbl>
1 loop          4.81ms  6.42ms    133.    4.83MB    20.2
2 vectorized    1.03ms  1.63ms    480.   167.95KB     1.93

```

```
summary(results, relative = TRUE)
```

```
# A tibble: 2 × 6
  expression    min median `itr/sec` mem_alloc `gc/sec`
  <bch:expr> <dbl>   <dbl>     <dbl>     <dbl>   <dbl>
1 loop         4.67   3.94         1       29.5    10.5
2 vectorized    1     1         3.62         1         1
```

Vectorizing the Bayesian bootstrap nets massive memory savings and a mild speedup in wall clock time, though note that the routine only takes a few milliseconds so in absolute terms the speedup probably won't be extremely noticeable.

Similarly, though less interesting, is a gradient calculation in `stacking_weights()`, just above the previous function. `stacking_weights()` is similarly used for combining models using stacking, that is, fitting a linear model to weigh the contending models. The gradient function is used for solving for those weights. However, the speedup here is less interesting because we are looping over K , the number of models being compared, which is usually small. Still, we may as well take the (potential) free performance. This is implemented in [#370](#).

```
N <- 400
K <- 20
lpd_point <- matrix(rnorm(N * K, mean = -1), nrow = N, ncol = K)
w <- rep(1 / K, K - 1)

stacking_gradient_loop <- function(w, lpd_point) {
  K <- ncol(lpd_point)
  w_full <- c(w, 1 - sum(w))
  grad <- rep(0, K - 1)
  mlpd <- matrixStats::rowMaxs(lpd_point)

  for (k in 1:(K - 1)) {
    grad[k] <- sum(
      (exp(lpd_point[, k] - mlpd) -
       exp(lpd_point[, K] - mlpd)) /
      exp(
        matrixStats::rowLogSumExps(sweep(lpd_point, 2, log(w_full), '+')) -
        mlpd
      )
    )
  }

  -grad
}

stacking_gradient_vectorized <- function(w, lpd_point) {
```

```

K <- ncol(lpd_point)
w_full <- c(w, 1 - sum(w))
mlpd <- matrixStats::rowMaxs(lpd_point)
denom <- exp(
  matrixStats::rowLogSumExps(sweep(lpd_point, 2, log(w_full), '+')) - mlpd
)
grad <- colSums(
  (exp(lpd_point[, 1:(K - 1), drop = FALSE] - mlpd) -
    exp(lpd_point[, K] - mlpd)) /
    denom
)

-grad
}

results <- bench::mark(
  loop = stacking_gradient_loop(w, lpd_point),
  vectorized = stacking_gradient_vectorized(w, lpd_point),
  iterations = 1e3
)

summary(results)

```

```

# A tibble: 2 × 6
  expression      min  median `itr/sec` mem_alloc `gc/sec`
  <bch:expr> <bch:tm> <bch:tm>    <dbl> <bch:byt>    <dbl>
1 loop        7.33ms 12.79ms   66.1    2.67MB     4.75
2 vectorized 629.6µs  1.02ms  679.    281.6KB     3.41

```

```
summary(results, relative = TRUE)
```

```

# A tibble: 2 × 6
  expression  min median `itr/sec` mem_alloc `gc/sec`
  <bch:expr> <dbl> <dbl>    <dbl>    <dbl>    <dbl>
1 loop      11.6 12.5      1      9.69     1.39
2 vectorized 1     1    10.3      1      1

```

The vectorized path is an order of magnitude faster, and uses about an order less memory as well. Again, like the previous change, it is important to note that these calls are very quick so an order faster may not even be noticeable in typical usage.

7 Nested parallelization

When updating `loo`, we wish to avoid “nested parallelization”, that is, attempting to compute something in parallel, when already in a parallel process. Take this nonfunctioning `mirai` code as an example:

```
# parallel over xs
mirai_map(
  xs,
  # parallel over x
  function(x) mirai_map(x, mean)[]
)
```

We first are doing something in parallel over values of `xs`, but then we try to work in parallel again for each value from `xs`! The main problem here is oversubscription: we spawn `n` outer workers, say, and then for each we spawn another `m` inner workers. In the package context, though, this implementation would be far more complex, and would probably be hidden completely from users. This becomes a problem when a user sets `n`, the number of outer parallel workers, to `# cores - 1`³, as the actual number of parallel tasks could be as large as $n * m$. Oversubscription is bad because, in this context, we are mostly running CPU-heavy tasks where each process takes time and needs its own core. Oversubscription is promising more parallelization than the hardware can provide—note that parallelization usually has some fixed startup costs as well (see Section 3.2), so we would probably lose performance. In `loo` specifically, we must be careful when parallelizing functions to ensure we do not accidentally have nested parallelization. Note that `mirai` exposes a function, `on_daemon()`, which may be helpful to avoid nested parallelization [3]. Further, note that calling `daemons(n)` from inside a `mirai_map()` worker errors rather than starting another local worker pool [3]. This removes one source of accidental nesting after migration. However, explicitly configured nested maps are still possible, so `loo` still needs its own guards around inner parallel operations.

7.1 Q5: Where do we currently find nested parallelization in `loo`?

Within `loo`, nested parallelization can arise where a function already being evaluated inside a parallel map calls another function that also parallelizes. Most of the internal paths where this could happen already pass `cores = 1` to the inner operation, so here we highlight those guards and one place where an attempted guard does not work.

In `loo.function()`, `.loo_i()` runs inside the map over observations and calls `importance_sampling.matrix()` with `cores = 1`. `loo_approximate_posterior.function()` follows the same pattern: `.loo_ap_i()` runs inside the observation-level map and calls `ap_psis()` with `cores = 1`.

³This is a common heuristic for number of threads, daemons, workers, etc. to use because it leaves one core free to do normal computer things, lest the system become unresponsive. Note that this doesn’t account for memory usage at all—if you have 8 GB memory but send out `n` tasks, each using 2 GB, you may run out of memory (see Section 4).

Moment matching similarly maps `loo_moment_match_i()` over observations above `k_threshold`. Inside each worker, its direct `relative_eff()` calls and its call to `loo_moment_match_split()` use `cores = 1`, keeping those inner `loo` operations serial. It also passes `cores = 1` through `update_quantities_i()` to the user-supplied `log_prob_upars()` and `log_lik_i_upars()` functions through `...`. This is another way `loo` tries to avoid nested parallelization when calling user code, though it ultimately cannot enforce whether those functions parallelize internally.

`relative_eff.function()` also attempts to keep its inner work serial by calling `relative_eff.default(..., cores = 1)` inside each observation-level worker. However, `relative_eff.default()` does not forward `...` to `relative_eff.matrix()`, so the supplied `cores = 1` is dropped. `relative_eff.matrix()` instead falls back to `getOption("mc.cores", 1)` and passes that value to `relative_eff.array()`, where the serial or parallel path is selected.

7.2 Can packages that use `loo` cause nested parallelization?

However, we know that other packages also call `loo` functions, which is another potential vector for nested parallelization. To this end, it seems like `brms` is the only (Stan) package where this could even be vaguely possible, but doing so seems to necessitate willful parallelization not provided natively by `brms`. This merits further investigation, but it seems like we can (softly) table concerns regarding external nested parallelization for now.

- `brms`
 - No issues.
 - `cores` appears nowhere in `R/loo.R`; `prepare_loo_args()` passes `...` straight through, so `cores` only reaches `loo()` if the user supplies it.
- `rstanarm`
 - No issues.
 - `kfold.stanreg()` explicitly avoids nested parallelism. With `cores > 1`, it parallelizes across folds and forces each Stan refit to use one core. With `cores = 1`, folds are fitted sequentially and each refit may instead use `getOption("mc.cores")` cores for its MCMC chains.
- `projpred`
 - No issues.
 - Pins `loo` calls inside the cross-validation loop to `cores = 1`.

References

- [1] R Core Team, *R: A language and environment for statistical computing*. Vienna, Austria: R Foundation for Statistical Computing, 2026. Available: <https://www.R-project.org/>
- [2] A. Vehtari, D. Simpson, A. Gelman, Y. Yao, and J. Gabry, “Pareto smoothed importance sampling,” *Journal of Machine Learning Research*, vol. 25, no. 72, pp. 1–58, 2024, Available: <https://jmlr.org/papers/v25/19-556.html>
- [3] C. Gao, *Mirai: Minimalist async evaluation framework for r*. 2026. Available: <https://mirai-r-lib.org>
- [4] A. Vehtari *et al.*, “Loo: Efficient leave-one-out cross-validation and WAIC for bayesian models.” 2025. Available: <https://mc-stan.org/loo/>

- [5] C. Gao, *Mori: Shared memory for r objects*. 2026. Available: <https://shikokuchuo.net/mori/>
- [6] C. Gao, H. Wickham, D. Vaughan, and L. Henry, “Parallel processing in purrr 1.1.0,” July 10, 2025. Available: <https://tidyverse.org/blog/2025/07/purrr-1-1-0-parallel/>
- [7] H. Wickham and L. Henry, *Purrr: Functional programming tools*. 2026. Available: <https://purrr.tidyverse.org/>
- [8] W. M. Landau, “Crew: A distributed worker launcher framework,” 2026. Available: <https://wlandau.github.io/crew/>
- [9] M. Kuhn, *Tune: Tidy tuning tools*. 2026. Available: <https://tune.tidymodels.org/>
- [10] C. Gao, *Nanonext: NNG (nanomsg next gen) lightweight messaging library*. 2026. Available: <https://nanonext.r-lib.org>
- [11] C. Gao, “Mirai 2.5.0,” Sept. 05, 2025. Available: <https://tidyverse.org/blog/2025/09/mirai-2-5-0/>
- [12] J. Hester and D. Vaughan, *Bench: High precision timing of r expressions*. 2025. Available: <https://bench.r-lib.org/>
- [13] H. Bengtsson, “A unifying framework for parallel and distributed processing in R using futures,” *The R Journal*, vol. 13, no. 2, pp. 208–227, 2021, doi: [10.32614/RJ-2021-048](https://doi.org/10.32614/RJ-2021-048)
- [14] H. Bengtsson, *Future.apply: Apply function to elements in parallel using futures*. 2026. Available: <https://future.apply.futureverse.org>
- [15] H. Bengtsson, “A unified approach to concurrent, parallel map-reduce in R using futures.” arXiv, Jan. 2026. doi: [10.48550/arXiv.2601.17578](https://arxiv.org/abs/2601.17578). Available: <https://arxiv.org/abs/2601.17578>. [Accessed: June 02, 2026]
- [16] H. Bengtsson, *Future.mirai: A future API for parallel processing using mirai*. 2025. Available: <https://future.mirai.futureverse.org>
- [17] C. Gao, “Mori: Shared memory for r objects,” Apr. 23, 2026. Available: https://opensource.posit.co/blog/2026-04-23_mori-0-1-0/. [Accessed: May 26, 2026]
- [18] H. Wickham, *Advanced r*, 2nd ed. Chapman; Hall/CRC, 2019. doi: [10.1201/9781351201315](https://doi.org/10.1201/9781351201315). Available: <https://adv-r.hadley.nz/>

Appendix

Glossary

Concepts

Serial

Tasks run one at a time, strictly one after another with no overlap.

Synchronous

The calling code blocks and waits for a result before continuing. Does not imply serial as tasks may run in parallel while the caller waits. Example: Parallel sampling across four chains but before we can continue, we have to wait until all four chains are finished.

Asynchronous

The calling code moves on immediately after dispatching a task and can check back later for results. Enables non-blocking workflows.

Parallel

Multiple tasks run at the same time across separate workers. Can be combined with either synchronous or asynchronous calling patterns. Examples:

- Synchronous parallel: running four chains in parallel but blocking until all are done.
- Asynchronous parallel: dispatching multiple model fits to background processes. The main session fires off each model fit without blocking, so you could immediately dispatch the next model. Each background process then internally runs its four chains in parallel (sync).

Forking

A child process is created by copying the parent R session at the OS level. The child inherits the full environment (packages, variables) via copy-on-write memory. Unix/macOS only.

PSOCK cluster

Worker processes started as completely fresh R sessions, communicating with the controller over sockets. No shared memory; packages and variables must be exported explicitly.

Daemon

A persistent background R process that stays alive between tasks, waiting to receive and execute work. Used by [mirai](#) as its worker model.

Persistent workers

A worker process that remains alive between tasks and across multiple calls, retaining its state and loaded packages. Avoids the startup cost of spawning a new process per job. Contrast with temporary workers (e.g., forked processes in [mclapply\(\)](#)) that are created for a single call and exit when it finishes.

Functions

[lapply\(\)](#)

Base R serial map. Applies a function to each element of list-like input sequentially in the host session. No workers, no parallelization overhead; considered as the baseline.

[mclapply\(\)](#) (Unix only)

Parallel map via forking. Workers inherit the session environment automatically. Falls back to [lapply\(\)](#) (i.e., serial execution) on Windows.

parLapply() (all OS)

Parallel map over a PSOCK cluster. Requires explicit setup (`makeCluster`), export of variables, and teardown (`stopCluster`). Workers persist until stopped.

daemons()

Launches n persistent background workers for the current `mirai` session. Workers stay alive and idle until reassigned or the session ends. Workers can run on the same machine, or can be remote resources, and roughly can be considered as background R processes. See [the documentation](#) for more details.

mirai_map()

Parallel asynchronous map that dispatches tasks to daemons. Returns a list of unresolved `mirai` objects immediately. Append `[]` to block and collect all results.

[]

The collect operator on a `mirai` result. Blocks until all dispatched tasks are complete and returns their values.

Session Info

```
renv::dependencies(here::here("docs", "report.qmd"), quiet = TRUE)$Package |>
  unique() |>
  c("sessioninfo") |>
  setdiff("rmarkdown") |>
  sessioninfo::session_info(
    pkgs = _,
    info = c("platform", "packages"),
    dependencies = FALSE
  )
```

```
— Session info —————
setting  value
version  R version 4.6.1 (2026-06-24 ucrt)
os       Windows 11 x64 (build 26200)
system   x86_64, mingw32
ui       RTerm
language (EN)
collate   English_United States.utf8
ctype    English_United States.utf8
tz        America/Los_Angeles
date      2026-08-16
pandoc    3.10 @ c:\\Program Files\\Positron\\resources\\app\\quarto\\bin\\
\\tools/ (via rmarkdown)
quarto    1.10.18 @ C:\\PROGRA~1\\Quarto\\bin\\quarto.exe
```

— Packages

package	* version	date (UTC)	lib	source
bench	1.1.4	2025-01-16	[1]	RSPM (R 4.6.0)
cmdstanr	0.9.0	2025-03-30	[1]	https://stan-dev.r-universe.dev (R 4.6.1)
future	1.70.0	2026-03-14	[1]	RSPM (R 4.6.0)
future.apply	1.20.2	2026-02-20	[1]	RSPM (R 4.6.0)
future.mirai	1.0.0	2026-06-22	[1]	CRAN (R 4.6.1)
lobstr	1.2.1	2026-04-04	[1]	RSPM (R 4.6.0)
loo	2.10.1	2026-07-24	[1]	CRAN (R 4.6.1)
matrixStats	1.5.0	2025-01-07	[1]	RSPM (R 4.6.0)
mirai	2.7.1	2026-06-01	[1]	CRAN (R 4.6.1)
mori	0.2.1	2026-06-09	[1]	RSPM (R 4.6.0)
purrr	1.2.2	2026-04-10	[1]	CRAN (R 4.6.1)
sessioninfo	1.2.4	2026-06-04	[1]	RSPM (R 4.6.0)

[1] C:/Users/visru/AppData/Local/R/win-library/4.6

[2] C:/Program Files/R/R-4.6.1/library
